

Company Interview Playbook — Volume 2

Full technical & behavioral question banks with ideal answers, common mistakes, follow-ups, and evaluation criteria — companion volume covering Oracle, Alibaba Cloud, Amazon, Flipkart, Microsoft, Nvidia, Tesla, OpenAI, Broadcom, Adobe, and Spotify. Ends with a consolidated RAG-ready dataset table for direct ingestion into company_playbook_chunks.

Note on this document: technical and behavioral questions here are authored/synthesized to reflect each company's publicly known interview format, engineering focus areas, and stated values/leadership principles — treat them as high-fidelity practice material and directional guidance, not as leaked or officially confirmed verbatim questions.

Oracle

Oracle hires for deep fundamentals over trendiness: JVM internals, SQL/query-plan literacy, and distributed-systems basics for OCI (Oracle Cloud Infrastructure) roles. Interviewers are often 10+ year veterans of the codebase and probe for whether you actually understand what you wrote, not just whether it compiles. Panels are IC-level (career level 3-6, roughly equivalent to junior through staff), and OCI-specific teams add a systems-design round focused on multi-tenant isolation and durability.

Interview Process

1. Online Assessment (OA) -- 2 DSA problems, HackerRank-style, 90 minutes, medium difficulty, auto-scored.
2. Technical Phone Screen -- 1 interviewer, 45-60 min, one DSA problem plus CS fundamentals (OOP, DBMS, OS) follow-ups.
3. Virtual Onsite (3-4 rounds) -- 2 DSA/coding rounds, 1 SQL/DBMS-heavy round, 1 System Design or LLD round (system design added for SDE-2+).
4. Hiring Manager / Bar Raiser round -- behavioral, culture fit, ownership signals, sometimes a second opinion on borderline technical rounds.

Technical Questions

Given a large orders table, a query filtering on customer_id and order_date is slow despite an index on customer_id. Why, and how would you fix it?

Difficulty: Medium | Topic: SQL / Query Optimization

Why the interviewer asks it

Oracle's business is databases -- they want to see if you understand how the query planner actually uses indexes, not just that indexes 'make things fast'.

Ideal thought process

Think about selectivity: an index on customer_id alone still requires scanning every matching row and filtering order_date in memory. Consider a composite index, index order (equality column first, range column second), and whether the optimizer is even choosing to use the index (check EXPLAIN PLAN / cardinality estimates).

Ideal answer

Create a composite index on (customer_id, order_date) so the engine can seek directly to the customer's rows and then range-scan by date within that leaf range, instead of scanning all of a customer's rows and filtering date afterward. Verify with EXPLAIN PLAN that Oracle picks an INDEX RANGE SCAN rather than a full table scan, and check that statistics are fresh (stale stats make the optimizer distrust a good index).

Common mistakes

Saying 'just add more indexes' without discussing column order; forgetting that too many indexes hurt write performance; not mentioning EXPLAIN PLAN as the verification step.

Follow-up question

What if customer_id has very low cardinality (e.g., only 5 distinct values across 10M rows)? Would the index still help?

Evaluation criteria

Understands index leaf structure and column-order tradeoffs; can name the verification tool (EXPLAIN PLAN); reasons about cardinality, not just 'add an index'.

Explain the difference between synchronized, ReentrantLock, and ConcurrentHashMap for protecting shared state in Java, and when you'd choose each.

Difficulty: Medium | Topic: Java Concurrency

Why the interviewer asks it

Oracle owns the JVM. Concurrency correctness questions are a recurring filter for any Java-heavy team.

Ideal thought process

Compare a JVM-level intrinsic lock (synchronized) against an explicit, interruptible, tryLock-capable lock (ReentrantLock), against a lock-striped concurrent collection purpose-built for map access patterns.

Ideal answer

synchronized is simple, JVM-managed, and released automatically on exception, but offers no tryLock/timeout and can't be interrupted while waiting. ReentrantLock gives you tryLock(), lockInterruptibly(), and fairness policies, at the cost of manually unlocking in a finally block. ConcurrentHashMap avoids locking the whole map by striping locks (or using CAS on later JVMs) across segments/bins, so use it directly instead of wrapping a HashMap in your own synchronized block whenever the shared state is 'a map'.

Common mistakes

Forgetting to unlock ReentrantLock in a finally block; claiming ConcurrentHashMap is 'lock-free' (it isn't, fully); not knowing synchronized can't time out.

Follow-up question

How does ConcurrentHashMap achieve thread safety internally in Java 8+ compared to Java 7?

Evaluation criteria

Can articulate real tradeoffs (not just 'use concurrent classes'); understands lock release semantics and failure modes.

Design a connection pool for a database client library used by thousands of services inside Oracle Cloud.

Difficulty: Hard | Topic: System Design / LLD

Why the interviewer asks it

OCI teams need engineers who can reason about resource exhaustion and fairness at scale, not just CRUD APIs.

Ideal thought process

Clarify: max pool size, connection acquisition timeout, health-checking/eviction of stale connections, and what happens when the pool is exhausted (block, queue, or fail fast).

Ideal answer

Maintain a bounded blocking queue of live connections; `acquire()` polls the queue with a timeout and falls back to opening a new connection if under max size, else blocks or throws. A background reaper thread validates idle connections (ping) and evicts/replaces broken ones. Track per-caller acquisition metrics to catch leaks (a connection checked out and never returned). Expose configuration for min/max size, idle timeout, and validation query.

Common mistakes

Not handling connection leaks (checked out, never returned); ignoring thread-safety of the pool's internal queue; no timeout on acquisition leading to cascading blocked threads.

Follow-up question

How would you detect and recover from a leaked connection automatically?

Evaluation criteria

Considers failure modes (leaks, stale connections, exhaustion) unprompted; proposes concrete data structures, not hand-waving.

Behavioral Questions (STAR)

Tell me about a time you had to debug something you didn't write and had very little context on.

Situation

Set the scene: what was broken, who was affected, how much time pressure existed.

Action

Describe your systematic approach -- reproducing the issue, reading logs/stack traces, bisecting recent changes, forming and testing hypotheses one at a time.

Result

State the concrete fix and what you changed afterward (added logging, a test, documentation) so the next person has an easier time.

Poor answer (avoid this)

"I just kept trying random things until it worked." -- signals no systematic debugging process, a red flag for a company whose product IS a database engine other companies depend on.

Evaluation criteria

Looks for a repeatable debugging methodology and follow-through (did you leave the codebase better than you found it).

Describe a situation where you disagreed with a technical decision made by a senior engineer.

Situation

What was the decision, why did you disagree, what was at stake.

Action

How you raised the disagreement -- data-backed, respectful, in the right forum (not undermining them publicly).

Result

What happened: did you change their mind, did they convince you, or did you disagree-and-commit.

Evaluation criteria

Oracle values engineers who can push back with evidence but still execute once a decision is made -- looking for maturity, not conflict avoidance or stubbornness.

Alibaba Cloud

Alibaba Cloud interviews (for its international/global engineering orgs) lean heavily on distributed systems, networking fundamentals, and raw coding speed -- multiple rounds in a single day is common. Expect a strong emphasis on large-scale infrastructure (object storage, CDN, load balancing) given the company's core cloud business, plus a cultural round assessing hustle and adaptability.

Interview Process

1. Online Coding Test -- 2-3 algorithm problems, competitive-programming style, moderate to hard.
2. Technical Round 1 -- data structures & algorithms, live coding, whiteboard/shared editor.
3. Technical Round 2 -- computer networks + operating systems fundamentals, sometimes distributed systems design for cloud-infra roles.
4. System Design Round -- design a cloud-scale component (CDN, object store, load balancer).
5. HR / Culture Round -- career motivation, adaptability, willingness to work across time zones/teams.

Technical Questions

Given a large social graph, find the shortest connection path between two users (like 'degrees of separation').

Difficulty: Medium | Topic: Graphs / BFS

Why the interviewer asks it

Tests whether you reach for BFS (unweighted shortest path) rather than overcomplicating with Dijkstra, and whether you can scale it (bidirectional search) for huge graphs.

Ideal thought process

Unweighted shortest path is BFS. For very large graphs, single-direction BFS from one user explores an exponentially growing frontier -- bidirectional BFS from both ends meeting in the middle is dramatically cheaper.

Ideal answer

Run BFS from both the source and target simultaneously, alternating frontier expansion, and stop when the frontiers intersect. This reduces the search space from $O(b^d)$ to roughly $O(b^{(d/2)})$ for branching factor b and depth d . Use a visited-set per side to detect the meeting point.

Common mistakes

Using Dijkstra when edges are unweighted (unnecessary overhead); not considering bidirectional search for a graph with millions of nodes; forgetting to track parent pointers to reconstruct the path.

Follow-up question

How would you distribute this BFS across multiple machines if the graph doesn't fit on one?

Evaluation criteria

Recognizes BFS = unweighted shortest path immediately; proposes bidirectional search as the scaling optimization without being told to.

Explain consistent hashing and why it's used in distributed caches and load balancers.

Difficulty: Medium | Topic: Distributed Systems / Networking

Why the interviewer asks it

Alibaba Cloud's core products (CDN, object storage, load balancing) all depend on this exact concept for horizontal scaling.

Ideal thought process

Contrast with naive modulo hashing ($\text{hash}(\text{key}) \% N$), which reshuffles almost every key when N changes. Consistent hashing places both nodes and keys on a hash ring so only a fraction of keys move when a node joins/leaves.

Ideal answer

Map both servers and keys onto a hash ring (e.g., using a 32-bit hash space); a key is owned by the first server clockwise from its position. Adding or removing a server only affects the keys between it and its predecessor on the ring, not the entire keyspace. Use virtual nodes (multiple ring positions per physical server) to avoid uneven load distribution.

Common mistakes

Not mentioning virtual nodes (leads to hot-spotting with few physical servers); confusing this with simple modulo hashing; not explaining why minimizing key movement matters (cache invalidation storms).

Follow-up question

How many virtual nodes would you use for 10 physical servers, and why does that number matter?

Evaluation criteria

Explains the ring model correctly and independently raises virtual nodes as the load-balancing refinement.

Design a global CDN edge-caching layer that serves static assets with low latency worldwide.

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

Directly maps to Alibaba Cloud's CDN product line -- assesses whether you can reason about geo-distribution, cache invalidation, and origin-shielding.

Ideal thought process

Clarify: read-heavy workload, cache-hit ratio targets, TTL/invalidation requirements, and origin server protection (thundering herd on cache miss).

Ideal answer

Use anycast routing or DNS-based geo-routing to send users to their nearest edge PoP. Each edge node caches by content hash/URL with a TTL; on a miss, use request coalescing (single-flight) so concurrent misses for the same key trigger one origin fetch, not N. Add a mid-tier regional cache between edge and origin to shield origin from the full edge-node fanout. Support active invalidation via a pub/sub purge system for content updates.

Common mistakes

Ignoring the thundering-herd problem on cache miss; no mention of invalidation/purge strategy; treating all edge nodes as if they talk directly to origin with no shielding tier.

Follow-up question

How do you handle a viral piece of content that suddenly gets 100x normal traffic at one edge location?

Evaluation criteria

Addresses geo-routing, cache miss stampede protection, and invalidation as three separate concerns, not just 'cache it at the edge'.

Behavioral Questions (STAR)

Tell me about a time you had to deliver under a very tight deadline with incomplete requirements.

Situation

The ambiguity, the deadline, and the stakes.

Action

How you scoped down to a shippable subset, communicated assumptions, and iterated.

Result

What shipped, what you'd do differently, how stakeholders reacted.

Evaluation criteria

Alibaba's fast-moving culture rewards bias-to-action -- looking for someone who ships and adjusts rather than blocking on perfect clarity.

Describe how you've worked with a team across a significant time-zone or language gap.

Situation

The team structure and communication challenge.

Action

Concrete tactics: async documentation, overlap-hour scheduling, over-communication of context.

Result

The outcome and what you learned about remote/cross-cultural collaboration.

Evaluation criteria

Global engineering orgs specifically probe for this -- signals of someone who over-documents and doesn't assume shared context are viewed favorably.

Amazon

Amazon's interview loop is inseparable from its 16 Leadership Principles (LPs) -- every behavioral answer is silently scored against 2-3 specific LPs per interviewer, and a dedicated 'Bar Raiser' (an interviewer from outside the hiring team with veto power) sits in on the loop specifically to prevent quality dilution. Technical bar is solid-medium DSA plus LP-infused follow-ups even inside coding rounds ('tell me about a time this exact tradeoff bit you in production').

Interview Process

1. Online Assessment -- 2 coding problems + a work-style/behavioral simulation assessment.
2. Phone Screen -- 1 DSA problem + LP-based behavioral questions, 60 min.
3. Onsite Loop (4-5 rounds) -- 2-3 coding rounds (DSA + LP blend), 1 system design round (SDE-2+), 1 Bar Raiser round (deep LP probing, can override the panel).
4. Bar Raiser debrief & hiring decision -- panel meets, Bar Raiser has explicit veto power regardless of other interviewers' votes.

Technical Questions

Given a stream of purchase events, design a system to detect the top-K most purchased products in the last sliding hour, updated in real time.

Difficulty: Hard | Topic: Data Structures / Streaming

Why the interviewer asks it

This is essentially 'build a mini version of Amazon's own best-seller ranking' -- tests sliding-window + heap combined with real-time constraints.

Ideal thought process

Naive re-scan of the last hour on every query is too slow. Need a sliding window structure (e.g., bucketed counts per minute) plus a top-K structure (min-heap of size K or a count-min sketch for approximate counts at extreme scale).

Ideal answer

Bucket counts into 1-minute windows (60 buckets for a rolling hour); on each new event, increment the current bucket's counter for that product and expire the oldest bucket every minute. Maintain a min-heap of size K over aggregate counts, lazily rebuilt or incrementally updated on each bucket rotation. At extreme scale, replace exact counts with a Count-Min Sketch to bound memory at the cost of approximate counts.

Common mistakes

Re-scanning the whole event history per query (too slow); not addressing expiry of old data (unbounded memory growth); ignoring the accuracy-vs-memory tradeoff at scale.

Follow-up question

How would this change if the system needed to be exact, not approximate, at 10M events/second?

Evaluation criteria

Recognizes the sliding-window bucketing pattern and the heap/top-K pattern as two separate composable pieces.

You own a checkout service that occasionally double-charges customers during retries after network timeouts. How do you fix this?

Difficulty: Medium | Topic: Distributed Systems / Idempotency

Why the interviewer asks it

Directly mirrors Amazon's actual production concerns (Ownership + Customer Obsession LPs baked into a technical question).

Ideal thought process

The root cause is that a retry after a timeout can't distinguish 'the first request never arrived' from 'the first request succeeded but the response was lost'. The fix is making the operation idempotent.

Ideal answer

Require an idempotency key (client-generated UUID) on every charge request; the payment service stores a record of (idempotency_key -> result) and, on seeing a duplicate key, returns the cached result instead of re-executing the charge. Use a database unique constraint on the key to make this atomic under concurrent retries.

Common mistakes

Proposing 'just don't retry' (breaks reliability); not handling the race condition where two retries with the same key arrive concurrently; forgetting the unique constraint/atomicity requirement.

Follow-up question

What if the idempotency key store itself needs to survive a regional failover?

Evaluation criteria

Names 'idempotency key' unprompted and reasons about the atomicity of the dedup check itself, not just 'store a flag'.

Design Amazon Locker: a system where customers pick up packages from self-service lockers using a code.

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

A real Amazon internal-style problem -- assesses end-to-end system thinking: capacity allocation, code generation/security, and failure handling (locker full, code expired).

Ideal thought process

Clarify: locker inventory reservation before delivery, code generation and expiry, what happens if no locker is free nearby, and pickup verification.

Ideal answer

Reserve a locker slot at label-print time via a locker-inventory service (locker_id, size, status); generate a single-use, time-boxed pickup code tied to (order_id, locker_id) and send it to the customer. On pickup, validate code + expiry + locker_id server-side before physically unlocking (IoT command to the locker hardware). Add a fallback flow (extend hold time, reroute to nearest available locker) if the customer doesn't arrive within the SLA window.

Common mistakes

Not reserving locker capacity ahead of delivery (leads to 'locker full' at drop-off); weak code generation (predictable/reusable codes); no expiry/reroute logic for abandoned packages.

Follow-up question

How do you prevent someone from brute-forcing another customer's pickup code?

Evaluation criteria

Thinks through the full lifecycle (reserve -> deliver -> notify -> pickup -> expire) and raises security of the pickup code without prompting.

Behavioral Questions (STAR)

[Ownership] Tell me about a time you fixed a problem that wasn't technically your responsibility.

Situation

What was broken, who 'owned' it on paper, why you stepped in anyway.

Action

What you actually did -- did you just patch it, or fix the underlying process so it wouldn't recur.

Result

The outcome, and ideally a long-term system change, not just a one-off fix.

Poor answer (avoid this)

"I told the owning team and moved on." -- technically correct but reads as low-ownership at Amazon; they want to hear you closed the loop yourself.

Evaluation criteria

Amazon explicitly wants 'I own outcomes beyond my job description' -- vague or team-diffused answers ('we decided...') get flagged.

[Bias for Action] Describe a decision you made with incomplete data because waiting would have cost too much.

Situation

The decision, the missing information, the time pressure.

Action

How you assessed risk/reversibility before acting -- Amazon's own framing is 'most decisions are two-way doors (reversible)'.

Result

What happened, and whether it validated the bias-for-action approach.

Evaluation criteria

Interviewers specifically listen for reversibility reasoning -- did you recognize this was a low-risk, reversible bet before moving fast?

[Disagree and Commit] Tell me about a time you disagreed with a decision but committed to it anyway.

Situation

The decision and why you disagreed.

Action

How you voiced disagreement (data-backed, once, clearly) before the decision was finalized.

Result

How you executed post-decision despite disagreeing, and the outcome.

Evaluation criteria

The key discriminator: did you keep re-litigating the decision after it was made, or genuinely commit? Amazon penalizes answers that show passive resistance.

Flipkart

Flipkart's SDE loop is DSA-heavy in early rounds (very similar bar to Amazon/Microsoft India) with a distinctive 'machine coding' round -- you write working, compilable, testable object-oriented code for a mini-system (parking lot, rate limiter, splitwise-style expense sharer) in 60-90 minutes, judged on design quality and extensibility, not just correctness. System design rounds skew toward Indian e-commerce scale (flash sales, cart/inventory consistency).

Interview Process

1. Online Assessment -- 2 DSA problems + MCQs on CS fundamentals.
2. Technical Round 1 -- DSA, 2 problems, live coding.
3. Machine Coding Round -- design + implement a working mini-application (e.g., parking lot, rate limiter) in an IDE, 60-90 min, evaluated on OOP design and extensibility.
4. System Design Round -- large-scale design (catalog, cart, flash-sale inventory), for SDE-2+.
5. Hiring Manager Round -- behavioral, ownership, culture fit.

Technical Questions

Given an array of stock prices, find the maximum profit from at most two transactions.

Difficulty: Medium | Topic: Dynamic Programming

Why the interviewer asks it

A step up from the classic single-transaction 'best time to buy/sell stock' -- tests whether you can extend a greedy insight into a DP formulation.

Ideal thought process

Track four running states: max profit after 1st buy, 1st sell, 2nd buy, 2nd sell, updating them in a single left-to-right pass.

Ideal answer

Maintain $buy1 = \max(-price)$ so far, $sell1 = \max(sell1, price + buy1)$, $buy2 = \max(buy2, sell1 - price)$, $sell2 = \max(sell2, price + buy2)$, updating all four on each price in one $O(n)$ pass, $O(1)$ space. $sell2$ at the end is the answer.

Common mistakes

Trying to brute-force all transaction pairs ($O(n^2)$ or worse); not realizing the two-transaction case still admits an $O(n)$ single-pass solution; conflating 'at most two' with 'exactly two'.

Follow-up question

How would you generalize this to at most K transactions?

Evaluation criteria

Recognizes the state-machine/DP pattern quickly; can generalize to K transactions with a 2D DP table when asked.

Design and implement (working code) a rate limiter that allows at most N requests per user per minute.

Difficulty: Medium-Hard | Topic: Machine Coding / LLD

Why the interviewer asks it

This is the archetypal Flipkart machine-coding prompt -- clean OOP design, thread-safety, and extensibility (can you swap algorithms later) all get graded.

Ideal thought process

Pick an algorithm (sliding window log, fixed window counter, or token bucket) and design classes so the algorithm is pluggable behind an interface -- interviewers explicitly reward extensibility.

Ideal answer

Define a RateLimiter interface with allowRequest(userId), and implement it with a token bucket per user (capacity N, refill rate N/60s), stored in a ConcurrentHashMap. Each Bucket lazily refills tokens based on elapsed time on access rather than a background thread, and allowRequest() does a compare-and-swap style decrement guarded by a lock per bucket (not a global lock) to keep contention low.

Common mistakes

Using one global lock across all users (kills concurrency); hardcoding the algorithm instead of hiding it behind an interface; forgetting to handle clock/time-based refill correctly (off-by-one on window boundaries).

Follow-up question

How would you change this to a sliding-window-log algorithm without touching the calling code?

Evaluation criteria

Code compiles/is coherent, uses per-user locking (not global), and the design is extensible -- graders specifically try swapping the algorithm to see if only one class needs to change.

Design the cart and inventory system for a flash sale where 1 million users try to buy 10,000 units of the same item in the first 30 seconds.

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

This is Flipkart's signature system-design scenario (Big Billion Days-style flash sales) -- overselling and cart/inventory race conditions are the exact production incidents this question is drawn from.

Ideal thought process

Clarify: is overselling by even 1 unit acceptable (no), what's the consistency requirement at checkout vs at 'add to cart', and how do you handle the thundering-herd read/write spike.

Ideal answer

Decouple 'add to cart' (optimistic, no inventory lock) from 'checkout' (atomic inventory decrement). At checkout, use an atomic conditional decrement (UPDATE inventory SET qty = qty - 1 WHERE item_id = ? AND qty > 0, check rows affected) or a distributed counter/semaphore in Redis (DECR with a floor check) to prevent overselling under concurrency. Queue excess demand (virtual waiting room) so the backend never receives more concurrent checkout attempts than it can safely process, and pre-warm caches for the flash-sale item ahead of time.

Common mistakes

Using read-then-write inventory checks (classic race condition -> overselling); no queueing/throttling for the traffic spike; treating this as a pure caching problem without addressing the write-path race.

Follow-up question

How do you handle a user who added the item to their cart but doesn't check out for 10 minutes -- do you hold inventory for them?

Evaluation criteria

Explicitly names the overselling race condition and proposes an atomic decrement or distributed lock, not just 'cache it' or 'scale horizontally'.

Behavioral Questions (STAR)

Tell me about a time you took ownership of a feature end-to-end, from design through production support.

Situation

The feature, its scope, and your role.

Action

Design decisions, how you handled edge cases, and what you did post-launch (monitoring, on-call, iterating on feedback).

Result

Impact metrics if available, and what broke (if anything) and how you responded.

Evaluation criteria

Flipkart's hiring managers weight end-to-end ownership heavily -- answers that stop at 'I shipped it' without mentioning production behavior read as incomplete.

Describe a time you had to make a tradeoff between shipping fast and doing it 'the right way'.

Situation

The deadline pressure and the tension.

Action

How you decided, what corners you consciously cut (and documented as tech debt) vs. what you refused to compromise on.

Result

What happened, and whether you circled back to pay down the debt.

Evaluation criteria

Looking for conscious, documented tradeoffs -- not blind speed and not perfectionism that misses deadlines.

Microsoft

Microsoft's SDE loop uses a 'levels' system (59-68 roughly junior to principal) and is known for its 'growth mindset' cultural emphasis post-Satya Nadella -- interviewers explicitly probe for how you respond to being wrong or not knowing something, not just whether you get the right answer. Technical rounds mix DSA, low-level design (LLD), and for senior candidates, a dedicated system design round; the Hiring Manager (not a separate Bar Raiser) makes the final call using an 'As One' collaborative model with the panel.

Interview Process

1. Online Assessment / Resume Screen -- for campus hires, an OA with 2-3 DSA problems.
2. Technical Phone Screen -- 1 DSA problem, 45 min, live coding with a Microsoft engineer.
3. Onsite Loop (4 rounds) -- 2 DSA/coding rounds, 1 LLD (object-oriented design) round, 1 System Design round (senior roles).
4. 'As Appropriate' (AA) / Hiring Manager Round -- values, growth mindset, collaboration style; the HM makes the final hire/no-hire call with panel input.

Technical Questions

Given a stream of integers, design a data structure that supports adding a number and finding the median in $O(\log n)$.

Difficulty: Medium-Hard | Topic: Heaps / Data Structures

Why the interviewer asks it

A classic Microsoft favorite -- tests whether you know the two-heap pattern rather than re-sorting on every query.

Ideal thought process

Maintain a max-heap for the lower half of numbers and a min-heap for the upper half, keeping their sizes balanced within 1 of each other.

Ideal answer

Push new numbers into the max-heap (lower half) first, then move its top to the min-heap (upper half) to preserve ordering; rebalance so sizes differ by at most 1. The median is either the top of the larger heap, or the average of both tops when sizes are equal. Insert is $O(\log n)$; median lookup is $O(1)$.

Common mistakes

Re-sorting the whole array on every insert ($O(n \log n)$ per op); using a single heap instead of two; forgetting the rebalancing step, which breaks the $O(1)$ median lookup.

Follow-up question

How would you extend this to support removing an arbitrary element from the stream?

Evaluation criteria

Names the two-heap pattern immediately and correctly reasons about the rebalancing invariant.

Design a parking lot system in code: multiple levels, multiple vehicle types, and dynamic pricing.

Difficulty: Medium | Topic: Low-Level Design (LLD)

Why the interviewer asks it

Tests OOP fundamentals -- class hierarchy, interface segregation, and whether you over- or under-engineer.

Ideal thought process

Identify core entities (ParkingLot, Level, Slot, Vehicle, Ticket) and behaviors (assign slot by vehicle-size compatibility, compute fee on exit).

Ideal answer

Model Vehicle as an abstract base with subclasses (Motorcycle, Car, Truck) each declaring a required SlotSize. ParkingLot holds a list of Levels, each with Slots; a SlotAllocationStrategy interface (e.g., nearest-available) decides slot assignment, kept separate from ParkingLot so the strategy can change independently. On entry, issue a Ticket with entry timestamp and slot reference; on exit, a PricingStrategy interface computes the fee from duration and vehicle type, again decoupled so pricing rules can change without touching allocation logic.

Common mistakes

Hardcoding pricing/allocation logic inside the ParkingLot class instead of behind interfaces (violates open-closed principle); not modeling vehicle-to-slot size compatibility; over-engineering with unnecessary design patterns not justified by requirements.

Follow-up question

How would you support a 'reserve a slot in advance' feature without redesigning the core classes?

Evaluation criteria

Separates concerns behind interfaces (allocation vs. pricing) proactively; avoids both under-design (God class) and over-design (unjustified pattern soup).

Design a system that aggregates status/metrics from thousands of independent internal services into a single dashboard (like an internal Azure health dashboard).

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

Mirrors real Microsoft internal-tooling problems -- tests handling of partial failures and heterogeneous data sources at scale.

Ideal thought process

Clarify: push vs. pull model for metrics collection, freshness requirements, and what happens when a source service is itself down (shouldn't blank the whole dashboard).

Ideal answer

Use a pull-based collector that polls each service's /health and /metrics endpoints on a schedule, writing results to a time-series store; if a poll times out or fails, mark that service 'unknown/stale' rather than blocking the whole aggregation pipeline (bulkhead each source). Serve the dashboard from the time-series store (not live calls) so a slow or dead service can't degrade dashboard load time. Add alerting on staleness (no successful poll in N minutes) as its own signal.

Common mistakes

Making the dashboard call all services synchronously on page load (one dead service blocks the whole page); no distinction between 'service reports unhealthy' and 'we failed to reach the service at all'; no staleness alerting.

Follow-up question

How would you avoid the collector itself becoming a bottleneck as the number of monitored services grows 100x?

Evaluation criteria

Explicitly isolates failure of individual sources (bulkheading) and separates collection from serving -- doesn't propose a synchronous fan-out design.

Behavioral Questions (STAR)

Tell me about a time you received critical feedback that was hard to hear. How did you respond?

Situation

The feedback, the context, and your initial reaction.

Action

What you did with it -- did you get defensive, or did you genuinely investigate and change behavior.

Result

The concrete change and, ideally, later validation that it helped.

Poor answer (avoid this)

"I don't really get much negative feedback." -- reads as low self-awareness, directly working against the 'growth mindset' signal Microsoft explicitly screens for.

Evaluation criteria

This is Microsoft's single most-repeated behavioral theme post-2014 culture shift -- genuinely reflective answers score far higher than defensive or dismissive ones.

Describe a time you helped a struggling teammate succeed, even though it wasn't required of you.

Situation

Who was struggling and with what.

Action

What you actually did -- pairing, documentation, advocating for them, adjusting scope.

Result

The outcome for them and the team.

Evaluation criteria

Ties to Microsoft's 'make others great' leadership principle -- individual heroics without lifting others score lower than expected here.

Nvidia

Nvidia interviews are unusually hardware-aware for a software role -- even application-software candidates get questions probing memory hierarchy, parallelism, and (for many teams) C++ and/or CUDA fundamentals. Loops are typically team-specific rather than centralized, so depth varies by group (driver teams go deep on OS/kernel, ML infra teams go deep on distributed training, graphics teams go deep on graphics pipelines), but a hardware-aware systems-design round is close to universal for SDE roles.

Interview Process

1. Recruiter Screen + Resume Review -- often followed directly by a technical phone screen (less standardized OA than Amazon/Microsoft).
2. Technical Phone Screen -- 1-2 problems, DSA plus C/C++ fundamentals (pointers, memory management) for most SWE roles.
3. Team-Specific Onsite (3-5 rounds) -- mix of DSA, C++/systems programming, and for GPU/driver-adjacent roles, CUDA or parallel-computing concepts.
4. System Design / Architecture Round -- often hardware-aware (memory bandwidth, latency, parallelism) rather than generic web-scale design.
5. Hiring Manager / Team Fit Round -- since teams hire individually, this round weighs heavily on 'would this specific team want to work with you'.

Technical Questions

Explain warp divergence in CUDA and why it hurts performance.

Difficulty: Medium | Topic: CUDA / Parallel Computing

Why the interviewer asks it

A defining Nvidia-specific concept -- signals whether you understand SIMT execution, not just 'parallel programming' in the abstract.

Ideal thought process

GPUs execute threads in groups (warps, typically 32 threads) in lockstep -- all threads in a warp execute the same instruction at the same time.

Ideal answer

When threads within the same warp take different branches of an if/else (data-dependent branching), the GPU must serially execute both paths, masking off the threads not on the active path, instead of true parallel execution. This means the warp's effective throughput drops toward $1/(\text{number of divergent paths})$. Mitigate by restructuring code so branch conditions are warp-uniform (all threads in a warp take the same path) or by sorting/grouping data so divergence is minimized.

Common mistakes

Confusing warp divergence with general branch misprediction (a CPU concept); not explaining the masking/serialization mechanism; no mitigation strategy offered.

Follow-up question

How would you detect warp divergence in a real kernel using profiling tools?

Evaluation criteria

Correctly explains the lockstep/masking mechanism (not just 'branches are slow') and proposes a concrete mitigation.

You're optimizing a matrix multiplication kernel and memory bandwidth is the bottleneck, not compute. What do you do?

Difficulty: Hard | Topic: GPU Performance / Roofline Model

Why the interviewer asks it

Tests understanding of the roofline model (compute-bound vs. memory-bound) which underlies most real Nvidia performance-engineering work.

Ideal thought process

If arithmetic intensity (FLOPs per byte moved) is low, you're memory-bound regardless of how fast your compute units are -- the fix is increasing data reuse, not adding more compute.

Ideal answer

Use tiling/blocking: load a tile of the input matrices into fast on-chip shared memory once, then reuse it across many multiply-accumulate operations before evicting it, increasing arithmetic intensity per byte fetched from slower global memory. Also ensure memory accesses are coalesced (threads in a warp access contiguous addresses) to maximize achieved bandwidth. Check occupancy isn't the limiter separately -- shared memory tiling and coalescing are the two primary levers here.

Common mistakes

Proposing 'just use more threads/blocks' (doesn't fix a bandwidth-bound kernel); not mentioning shared memory tiling for data reuse; ignoring memory coalescing.

Follow-up question

How would you determine, using the roofline model, whether a kernel is compute-bound or memory-bound before optimizing?

Evaluation criteria

Correctly diagnoses low arithmetic intensity as the root cause and proposes tiling + coalescing, the two standard fixes, unprompted.

Design a hardware-aware system for scheduling thousands of ML training jobs across a cluster of GPUs with heterogeneous memory sizes.

Difficulty: Hard | Topic: System Design / ML Infra

Why the interviewer asks it

Reflects real Nvidia ML-infra work -- naive schedulers ignore that a job needing 80GB can't run on a 24GB GPU, wasting cluster capacity.

Ideal thought process

Clarify: jobs have declared or measured memory/compute requirements; goal is maximizing cluster utilization while respecting fairness across teams.

Ideal answer

Track each GPU's available memory and compute class in a scheduler's resource model; bin-pack jobs onto GPUs using best-fit on memory headroom rather than round-robin, and maintain separate queues per GPU class so small jobs don't get starved waiting for large-memory GPUs to free up. Add preemption/checkpointing for lower-priority jobs so high-priority jobs can claim scarce large-memory GPUs without waiting for natural completion.

Common mistakes

Treating all GPUs as interchangeable (ignores memory heterogeneity, causing OOM failures or wasted capacity); no fairness/priority mechanism; no preemption strategy for urgent jobs.

Follow-up question

How would you handle a job that needs multiple GPUs with fast interconnect (NVLink) between them specifically?

Evaluation criteria

Explicitly reasons about heterogeneous resource-matching (not just job count), and raises preemption/checkpointing without prompting.

Behavioral Questions (STAR)

Tell me about a time you had to learn a highly specialized or unfamiliar hardware/domain concept quickly to do your job.

Situation

What you didn't know and why it suddenly mattered.

Action

How you ramped up -- docs, pairing with a domain expert, hands-on experimentation.

Result

How it played out and what you'd do differently to ramp faster next time.

Evaluation criteria

Nvidia's domain (GPU architecture, CUDA, drivers) has a steep specialized learning curve -- this question filters for genuine intellectual curiosity vs. someone who'd stay in their comfort zone.

Describe a time you gave honest, critical feedback on a technical approach, even though it wasn't the popular opinion in the room.

Situation

The technical disagreement and the social dynamics at play.

Action

How you raised it -- data, benchmarks, respectful framing.

Result

What happened, whether it changed the outcome.

Evaluation criteria

Small, engineering-dense teams at Nvidia value engineers who'll say 'this benchmark doesn't support that claim' even to a senior person -- passive agreement is viewed as a weaker signal than respectful pushback.

Tesla

Tesla software interviews emphasize practical, ship-fast engineering over abstract whiteboard theory -- expect real-world coding tasks (sometimes take-home or IDE-based rather than pure algorithmic puzzles) and system-design questions that involve hardware-software co-design (vehicle software, energy systems, manufacturing/robotics software). Behavioral rounds probe hard for 'first-principles thinking' and tolerance for extremely high ownership and pace.

Interview Process

1. Recruiter Screen -- role fit, motivation, sometimes a basic technical gut-check.
2. Technical Phone Screen -- 1 coding problem (often more practical/applied than 'pure LeetCode'), plus fundamentals relevant to the team (embedded, backend, ML).
3. Onsite/Virtual Loop (3-5 rounds) -- coding round(s), a system-design round often involving real-time or hardware constraints, and a deep-dive on a past project.
4. Hiring Manager / Team Round -- heavy focus on ownership, pace, and first-principles problem solving; Tesla is known for a demanding, fast decision cycle.

Technical Questions

Given telemetry data streaming from a fleet of vehicles (location, battery %, speed) arriving out of order, design a way to detect a vehicle that has stopped unexpectedly (potential breakdown).

Difficulty: Medium-Hard | Topic: Streaming / Real-Time Systems

Why the interviewer asks it

Directly modeled on Tesla's actual fleet-telemetry problem space -- tests handling out-of-order events and windowed anomaly detection.

Ideal thought process

Out-of-order arrival means you can't just compare 'latest event' to 'previous event' naively -- you need event-time windowing with some tolerance for lateness (watermarks).

Ideal answer

Key telemetry by vehicle_id and buffer events in a small time window (e.g., a few minutes) sorted by event timestamp, not arrival time, using a watermark to decide when a window is 'complete enough' to evaluate. Within a window, if speed stays near zero and location doesn't change for longer than an expected threshold (accounting for legitimate stops like charging or traffic), flag an anomaly. Emit late-arriving corrections as an update if a straggler event changes the verdict.

Common mistakes

Using arrival order instead of event-time ordering (produces false positives from network jitter); not distinguishing 'legitimately parked/charging' from 'broken down'; no watermark/lateness tolerance.

Follow-up question

How do you avoid false positives when a vehicle is stopped at a supercharger for 30 minutes?

Evaluation criteria

Explicitly separates event-time from arrival-time and proposes a watermarking or lateness-tolerance mechanism unprompted.

Design the over-the-air (OTA) software update system for a fleet of millions of vehicles, ensuring a bad update can't brick cars.

Difficulty: Hard | Topic: System Design / Hardware-Software Co-design

Why the interviewer asks it

This is a genuine, safety-critical Tesla engineering problem -- tests staged rollout thinking and failure-recovery design, not just 'push files to devices'.

Ideal thought process

Clarify: rollback capability is non-negotiable (a bricked vehicle is a safety issue), and rollout must be staged/canaried, not all-at-once.

Ideal answer

Use an A/B partition scheme on-vehicle: the new firmware installs to the inactive partition while the vehicle keeps running the known-good active partition; only after a successful boot-and-health-check does the vehicle switch the active partition, with automatic rollback to the previous partition if the new one fails to boot or fails a post-boot self-test. Roll updates out in stages (small canary fleet, monitor telemetry for anomalies, then wider rollout) rather than pushing to the entire fleet simultaneously.

Common mistakes

Pushing to the whole fleet at once (no canary stage); overwriting the only working firmware image in place (no safe rollback path); no automated health-check gating the partition switch.

Follow-up question

How would you decide, using fleet telemetry, when it's safe to proceed from the canary stage to full rollout?

Evaluation criteria

Names A/B partitioning (or equivalent safe-rollback mechanism) and staged/canary rollout as two distinct, necessary pieces -- 'just push the update' answers fail this question.

Behavioral Questions (STAR)

Tell me about a time you questioned an existing process or requirement and it turned out to be unnecessary (first-principles thinking).

Situation

The process/requirement and why it seemed suspect.

Action

How you investigated the underlying reason it existed rather than just accepting 'that's how it's done'.

Result

What you changed and the impact.

Evaluation criteria

Tesla explicitly hires for first-principles reasoning (a stated Elon Musk engineering philosophy) -- answers that show 'I traced this back to its root assumption and it didn't hold' score highly; answers that just describe following instructions well score low.

Describe a time you had to move extremely fast on something with real consequences if you got it wrong.

Situation

The stakes and the time constraint.

Action

How you balanced speed against rigor -- what corners you refused to cut vs. what you accepted as acceptable risk.

Result

The outcome and what it taught you about pace under pressure.

Evaluation criteria

Tesla's pace is a known cultural extreme -- interviewers are checking whether you thrive under it or would burn out/freeze.

OpenAI

OpenAI's engineering interviews blend strong practical coding ability with genuine understanding of modern ML systems (even for non-research engineering roles) and unusually direct mission-alignment probing given the company's safety-focused charter. The loop is comparatively lean (fewer, denser rounds) and increasingly includes a paid, multi-day work-trial-style round for senior hires, reflecting a preference for evaluating real working style over interview performance alone.

Interview Process

1. Recruiter Screen -- background, motivation, and often a direct question about your view on AI safety/mission alignment.
2. Technical Screen -- 1-2 practical coding problems, often closer to 'build a small working system' than abstract algorithms.
3. Technical Onsite (2-3 rounds) -- coding, systems/ML-infra design, and for many roles a focused round on LLM-specific concepts (attention, RAG, evaluation).
4. Work Trial / Deep-Dive (senior roles) -- a paid, multi-day (sometimes up to 48-hour) collaborative work sample with the actual team.
5. Leadership / Mission Round -- values alignment, how you'd handle safety-vs-speed tradeoffs.

Technical Questions

Design a resumable iterator/checkpoint system for processing a multi-terabyte dataset that must survive a worker crash without reprocessing everything.

Difficulty: Medium-Hard | Topic: Systems / Data Engineering

Why the interviewer asks it

A direct proxy for real OpenAI infra work -- training data pipelines routinely process datasets far larger than a single run's lifetime and must tolerate crashes.

Ideal thought process

The key requirement is durable progress tracking that's cheap to write frequently but lets you resume from close to the failure point, not from zero.

Ideal answer

Partition the dataset into fixed-size shards up front; persist a checkpoint record (shard_id, offset_within_shard) to durable storage after each shard (or every N records for large shards) completes. On restart, read the last checkpoint and resume from that shard/offset rather than the beginning, and make each shard's processing idempotent (safe to reprocess the tail of a partially-done shard) so a crash mid-shard doesn't corrupt output.

Common mistakes

Checkpointing too infrequently (loses too much progress on crash) or too frequently (I/O overhead dominates); not making shard processing idempotent, so resuming mid-shard produces duplicate/corrupt output; storing checkpoints only in memory (lost on crash, defeats the purpose).

Follow-up question

How would you parallelize this across many workers while keeping checkpointing consistent?

Evaluation criteria

Proposes durable, granular checkpointing and explicitly addresses idempotency of resumed work, not just 'save progress somewhere'.

Explain, at a conceptual level, why the attention mechanism in Transformers scales quadratically with sequence length, and one practical approach to mitigate it.

Difficulty: Medium | Topic: ML Concepts / Transformers

Why the interviewer asks it

OpenAI expects even non-research engineers touching model infra to have working knowledge of why long-context serving is expensive.

Ideal thought process

Self-attention computes a similarity score between every pair of tokens in the sequence, which is inherently an n -by- n computation.

Ideal answer

Every token attends to every other token, producing an $n \times n$ attention matrix, so both compute and memory scale as $O(n^2)$ with sequence length n -- doubling context length roughly quadruples the attention cost. Practical mitigations include sparse/local attention (each token only attends to a nearby window plus a few global tokens), or KV-caching combined with techniques like sliding-window attention or FlashAttention, which restructures the computation to reduce memory movement (not the $O(n^2)$ compute itself, but the dominant real-world bottleneck).

Common mistakes

Confusing FlashAttention (a memory-I/O optimization) with actually reducing the $O(n^2)$ compute complexity; not being able to explain why it's quadratic in the first place (every-pair comparison); vague answers like 'attention is just expensive'.

Follow-up question

Why does KV-caching help with generation latency specifically, and what does it NOT solve?

Evaluation criteria

Correctly attributes the quadratic cost to all-pairs comparison and distinguishes compute-complexity fixes from memory-bandwidth fixes (FlashAttention) rather than conflating them.

Design a rate-limiting and fair-usage system for an API serving a very expensive-to-compute model to millions of developers with wildly different usage patterns.

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

This is literally OpenAI's own production problem (the API platform) -- tests fairness-under-scarcity thinking, not generic rate limiting.

Ideal thought process

Clarify: the scarce resource is GPU compute, not just request count, so rate limiting purely by request-count misses the point -- a 4K-token request costs far more than a 50-token one.

Ideal answer

Rate-limit on a token/compute-cost budget per account per time window (not raw request count), using a token-bucket where each request consumes tokens proportional to its estimated compute cost (input + expected output length). Tier limits by account trust/spend level, and add a separate, smaller shared 'burst' pool with priority queuing so no single heavy user can starve the shared capacity during a spike. Surface clear 429 responses with retry-after guidance so clients can back off gracefully.

Common mistakes

Rate-limiting purely by request count (ignores that request cost varies hugely by token length); no tiering by account trust or usage plan; no fairness mechanism to prevent one heavy client from starving others during a capacity crunch.

Follow-up question

How would you handle a sudden 10x demand spike for a viral use case without completely locking out existing paying customers?

Evaluation criteria

Explicitly ties rate limiting to compute cost (not request count) and proposes fairness/priority mechanisms for scarcity, not just a flat per-account cap.

Behavioral Questions (STAR)

Tell me about a time you had to balance moving fast on a feature against a safety, privacy, or correctness concern.

Situation

The tension and the stakes involved.

Action

How you weighed the tradeoff -- did you escalate, slow down, find a way to de-risk while still shipping.

Result

The outcome and whether it validated your judgment.

Evaluation criteria

Given OpenAI's stated mission, this question directly probes whether a candidate defaults to 'ship it' or genuinely internalizes safety/correctness tradeoffs -- answers that show zero hesitation ever are viewed skeptically.

Describe a time you changed your mind about a strongly-held technical opinion after seeing new evidence.

Situation

The original opinion and why you held it.

Action

What evidence or argument changed your view, and how you updated publicly (not just privately).

Result

The outcome and what it says about your epistemic habits.

Evaluation criteria

OpenAI's research-adjacent culture rewards visible, fast belief-updating -- rigid answers ('I've never really changed my mind on something I felt strongly about') are a negative signal here specifically.

Broadcom

Broadcom's software interviews (spanning networking chips, storage, and enterprise software divisions post-VMware acquisition) emphasize low-level systems knowledge -- operating systems, networking protocols, and C/C++ -- more than many 'pure app' companies, reflecting its semiconductor and infrastructure-software roots. Loops are lean and efficiency-focused; expect direct, no-frills technical rounds and a strong emphasis on debugging skill.

Interview Process

1. Recruiter/Resume Screen -- role and division fit (chip-adjacent software vs. enterprise software vary significantly).
2. Technical Phone Screen -- 1 DSA problem plus OS/networking fundamentals questions.
3. Onsite Loop (3-4 rounds) -- coding round(s), a systems/networking-fundamentals round, and a design round scoped to the division (e.g., a networking control-plane feature).
4. Hiring Manager Round -- team fit, debugging war stories, and practical delivery track record.

Technical Questions

Explain the difference between TCP and UDP, and describe a scenario where you'd choose UDP despite it being 'unreliable'.

Difficulty: Easy-Medium | Topic: Networking Fundamentals

Why the interviewer asks it

Core to Broadcom's networking-hardware business -- basic protocol literacy is a hard filter, not a nice-to-have.

Ideal thought process

TCP trades latency and overhead for guaranteed, ordered delivery via handshakes, acknowledgments, and retransmission; UDP has none of that overhead.

Ideal answer

TCP establishes a connection (3-way handshake), guarantees ordered delivery, and retransmits lost packets, at the cost of higher latency and per-connection overhead. UDP is connectionless and 'fire and forget' -- no guarantees, but minimal overhead and latency. Choose UDP for real-time applications where a late packet is worse than a lost one (live video/voice calls, some multiplayer game state, DNS queries) since retransmitting a stale packet is often pointless.

Common mistakes

Saying UDP is 'just worse than TCP' without identifying use cases where reliability isn't the priority; not mentioning the handshake/overhead tradeoff explicitly.

Follow-up question

How does QUIC try to get UDP's low latency with some of TCP's reliability guarantees?

Evaluation criteria

Names concrete UDP use cases (real-time media, DNS) with the correct underlying reasoning (staleness > loss for these), not just protocol trivia recall.

You're debugging a network driver that intermittently drops packets under high load but works fine in normal testing. How do you approach it?

Difficulty: Hard | Topic: OS / Debugging

Why the interviewer asks it

Reflects Broadcom's chip/driver-adjacent engineering reality -- tests systematic debugging under conditions that are hard to reproduce.

Ideal thought process

Intermittent, load-dependent bugs usually point to race conditions, buffer exhaustion, or resource limits that only manifest under contention -- reproduction strategy matters as much as the fix.

Ideal answer

First try to make the bug reproducible on demand by simulating high load in a controlled environment (load-testing tools, artificially throttled buffers) rather than debugging in production. Check for buffer/queue exhaustion (ring buffer overflow, dropped packets logged at the driver or NIC level), race conditions in interrupt handlers or DMA descriptor management, and use kernel tracing tools (e.g., ftrace, perf, or vendor-specific packet counters) to correlate drops with specific code paths. Narrow down via bisection of recent driver changes if this is a regression.

Common mistakes

Jumping straight to code changes without first establishing reliable reproduction; not considering race conditions/interrupt-context bugs specifically (a very common root cause for 'works under light load, fails under heavy load'); ignoring hardware-level counters/tracing tools available for exactly this class of bug.

Follow-up question

What's different about debugging a race condition in an interrupt handler versus a regular user-space thread?

Evaluation criteria

Prioritizes reliable reproduction first and explicitly considers race conditions/resource exhaustion as the likely root cause class for load-dependent bugs.

Design a control-plane service that pushes configuration changes to thousands of network switches, ensuring no switch ends up in an inconsistent state if the push fails partway.

Difficulty: Hard | Topic: System Design / Networking

Why the interviewer asks it

Mirrors real enterprise-networking software problems at Broadcom -- partial-failure handling for infrastructure config pushes is a recurring theme.

Ideal thought process

Clarify: is eventual consistency acceptable, or must all switches apply the change atomically (all-or-nothing)? Network partitions during rollout are a given at this scale.

Ideal answer

Stage the new config on each switch without activating it (two-phase: distribute, then commit), verify all reachable switches have successfully staged the config, and only then send a commit signal; switches that didn't ack the stage phase are excluded from commit and flagged for retry/alerting rather than left partially applied. Add a rollback path (previous config kept available) so any switch that fails post-commit health checks can revert automatically.

Common mistakes

Pushing and activating config in a single step (a mid-rollout failure leaves some switches on old config, some on new, with no clean recovery path); not handling unreachable switches explicitly (silent partial failure); no automatic rollback for a switch that becomes unhealthy after the new config activates.

Follow-up question

How would you handle a switch that acked the commit but is now unreachable for health checks -- do you assume success or failure?

Evaluation criteria

Proposes a two-phase stage/commit pattern with explicit handling of partial acks and automatic rollback, not a single-step 'push config to all switches' design.

Behavioral Questions (STAR)

Tell me about the hardest bug you've ever debugged. What made it hard, and how did you eventually solve it?

Situation

Symptoms, why it was hard to reproduce or isolate.

Action

Your systematic process -- tools used, hypotheses tested and discarded.

Result

The fix and what changed afterward to prevent recurrence.

Evaluation criteria

Broadcom's infrastructure-heavy roles weight this question unusually heavily -- vague or short answers here are a stronger negative signal than at more app-layer-focused companies.

Describe a time you had to work closely with a hardware or firmware team as a software engineer.

Situation

The cross-functional challenge and communication gap.

Action

How you bridged the gap -- learning enough of their domain, translating requirements both ways.

Result

The outcome and what it taught you about hardware-software collaboration.

Evaluation criteria

Many Broadcom software roles sit directly adjacent to hardware teams -- this checks for genuine comfort operating across that boundary, not just pure software isolation.

Adobe

Adobe's SDE interviews are DSA-solid but distinctly product- and craft-oriented -- system design rounds often involve real-time collaboration (Creative Cloud, Document Cloud) and large binary/media asset handling rather than generic CRUD services. Adobe explicitly evaluates candidates against its stated values (Genuine, Exceptional, Innovative, Involved) in behavioral rounds, and cares about UX/product sensibility even in pure backend roles.

Interview Process

1. Online Assessment -- 2 DSA problems, moderate difficulty, 60-90 min.
2. Technical Phone Screen -- 1 DSA problem plus CS fundamentals (OOP, DBMS).
3. Onsite Loop (3-4 rounds) -- 1-2 DSA rounds, 1 system/LLD design round often product-flavored (e.g., real-time document sync), 1 round blending technical depth with product sensibility.
4. HR / Values Round -- explicitly mapped to Adobe's stated values (Genuine, Exceptional, Innovative, Involved).

Technical Questions

Given a singly linked list, detect if it contains a cycle, and if so, find the node where the cycle begins.

Difficulty: Medium | Topic: Linked Lists

Why the interviewer asks it

A perennial Adobe favorite for filtering fundamentals -- tests both the detection (easy part) and the harder mathematical insight for locating the cycle start.

Ideal thought process

Floyd's cycle detection (slow/fast pointers) finds whether a cycle exists in $O(1)$ space; finding the start requires a second phase using the meeting point.

Ideal answer

Move slow by 1 and fast by 2 steps; if they meet, a cycle exists. To find the cycle's start, reset one pointer to the head and advance both pointers by 1 step at a time from the meeting point and head simultaneously -- they'll meet exactly at the cycle's starting node, a consequence of the distance math between head, cycle start, and meeting point.

Common mistakes

Using a hash set for $O(n)$ space when $O(1)$ space (Floyd's) is expected as the follow-up; not being able to justify why the second phase (reset-and-advance) correctly finds the cycle start, just reciting it as a memorized trick.

Follow-up question

Can you prove why resetting one pointer to head and advancing both by 1 finds the exact cycle start?

Evaluation criteria

Can derive or at least clearly explain the math behind phase two, not just recite the algorithm from memory.

Design the backend for a real-time collaborative document editor (like multiple users editing a PDF or design file simultaneously).

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

Maps directly onto Adobe's Creative Cloud / Document Cloud products -- real-time conflict resolution is a defining Adobe engineering challenge.

Ideal thought process

Clarify: is this text-like content (amenable to Operational Transformation / CRDTs) or binary/vector content (layers, shapes) where conflict resolution is trickier.

Ideal answer

Use a CRDT (Conflict-free Replicated Data Type) or Operational Transformation layer so concurrent edits from multiple clients merge deterministically without a central lock, broadcasting deltas (not full document state) over WebSocket connections to all connected clients. Maintain a canonical document state server-side that applies incoming operations in a consistent order, and periodically persist snapshots (not just the operation log) so reconnecting clients don't need to replay the entire edit history. Handle offline edits by queuing local operations and reconciling on reconnect.

Common mistakes

Proposing a simple 'last write wins' or full-document locking approach (breaks true real-time collaboration, causes edit loss); sending full document state on every change instead of deltas (doesn't scale); no offline/reconnect handling.

Follow-up question

How does OT differ from CRDTs, and which would you pick for a design tool with layered vector shapes versus a plain-text editor?

Evaluation criteria

Names CRDT or OT specifically (not vague 'sync it somehow') and reasons about delta-based updates plus offline reconciliation without prompting.

Design a service that generates and serves thumbnail previews for millions of large image/PDF/video assets uploaded to Creative Cloud, on demand and at scale.

Difficulty: Hard | Topic: System Design / Media Processing

Why the interviewer asks it

A direct proxy for Adobe's actual asset-pipeline problem -- balances async processing, caching, and cost control for compute-heavy transformations.

Ideal thought process

Clarify: generate thumbnails eagerly (on upload) vs. lazily (on first request), and how to avoid regenerating the same thumbnail repeatedly under concurrent requests.

Ideal answer

On upload, enqueue an async thumbnail-generation job (common sizes generated eagerly since they're almost always requested); for less common sizes, generate lazily on first request and cache the result (CDN + object storage) so subsequent requests are served without reprocessing. Use request coalescing so concurrent first-requests for the same missing thumbnail trigger one generation job, not N duplicate jobs. Separate the processing worker pool by asset type/cost (video transcoding is far more expensive than image resizing) so one type doesn't starve the other.

Common mistakes

Generating every possible thumbnail size eagerly on upload (wastes massive compute/storage for rarely-requested sizes); not handling the thundering-herd case of concurrent requests for an uncached thumbnail; treating image and video processing as the same cost class.

Follow-up question

How would you prioritize regenerating thumbnails if you changed your thumbnail format/quality settings for all 500 million existing assets?

Evaluation criteria

Distinguishes eager vs. lazy generation deliberately and addresses request coalescing for cache misses, not just 'cache the thumbnails'.

Behavioral Questions (STAR)

[Genuine / Involved] Tell me about a time you gave a teammate honest feedback that was uncomfortable to deliver.

Situation

The context and why the feedback was hard to give.

Action

How you framed it -- direct but constructive, timed appropriately.

Result

How it was received and the outcome for the relationship/project.

Evaluation criteria

Maps to Adobe's 'Genuine' value -- answers that avoid the discomfort entirely ('I just let it go') score lower than ones showing respectful directness.

[Innovative] Describe a time you proposed a creative or unconventional solution that others initially pushed back on.

Situation

The problem and why the obvious solution wasn't good enough.

Action

How you developed and pitched the unconventional idea, and how you handled pushback.

Result

Whether it was adopted, and the impact.

Evaluation criteria

Adobe explicitly values idea generation beyond 'safe, obvious' engineering -- answers here are compared against the 'Innovative' value pillar specifically.

Spotify

Spotify's engineering interviews are comparatively lightweight on pure LeetCode grinding and heavier on system design (streaming-specific: recommendation delivery, real-time playback state, catalog scale) and a distinctive 'case study' or take-home round assessing product judgment and communication. The 'squad' autonomous-team culture means behavioral rounds probe hard for collaborative, low-ego teamwork and giving/receiving feedback well.

Interview Process

1. Recruiter Screen -- background and motivation, role fit within a specific squad/tribe.
2. Technical Screen -- 1 DSA/coding problem, moderate difficulty, plus general engineering fundamentals discussion.
3. System Design Round -- streaming/catalog-scale design problem.
4. Case Study / Take-Home -- a product-and-engineering judgment exercise (e.g., 'how would you improve X feature and what would you measure'), often presented back to the panel.
5. Culture / Team Fit Round -- collaboration style, feedback culture, autonomy and alignment within a squad model.

Technical Questions

Given a playlist of songs, implement an efficient 'true shuffle' that never repeats a song until all songs have played, and can resume correctly if the app restarts.

Difficulty: Medium | Topic: Data Structures / Algorithm Design

Why the interviewer asks it

A real, product-grounded Spotify problem -- users have famously complained about shuffle 'feeling non-random' or repeating songs, so this tests both algorithmic correctness and product awareness.

Ideal thought process

A true derangement-free shuffle without repeats until exhaustion is just a shuffled permutation of the playlist (Fisher-Yates), with a persisted cursor/position so a restart resumes rather than reshuffles.

Ideal answer

Generate a shuffled order of song indices using Fisher-Yates ($O(n)$, uniform random permutation) at the start of a shuffle session, and persist both the shuffled order and the current playback index (not just 'shuffle mode: on'). On app restart, reload the persisted order and index rather than regenerating a new shuffle, so playback resumes exactly where it left off. When the playlist itself changes (song added/removed) mid-session, decide and communicate a policy (e.g., append new songs to the remaining unplayed portion).

Common mistakes

Using `random.choice()` repeatedly with only 'avoid last-played song' logic (produces clustering/repeats that users perceive as non-random, the actual real-world Spotify complaint); not persisting the shuffle order itself, only a 'shuffle on/off' flag, which breaks resume-after-restart.

Follow-up question

How do you handle mid-playlist edits (a song is added or removed) without breaking the persisted shuffle order?

Evaluation criteria

Recognizes Fisher-Yates as the correct primitive and explicitly persists order + cursor (not just a boolean flag), showing awareness of the real product complaint this question is based on.

Design the backend system that decides what to play next in real time as a user listens, accounting for personalized recommendations, without introducing playback lag.

Difficulty: Hard | Topic: System Design

Why the interviewer asks it

Directly maps to Spotify's core product experience (autoplay/radio) -- tests latency-aware design for a personalization system, not just 'call a recommendation model'.

Ideal thought process

Clarify: the recommendation model itself may be slow (ML inference), but playback can't stall waiting for it -- next-track decisions must be pre-computed ahead of the actual need.

Ideal answer

Pre-fetch and cache a short queue of candidate next-tracks (e.g., next 5-10) ahead of the current playback position, computed asynchronously by the recommendation service while the current track is still playing, so the decision is already made by the time it's needed. Refresh the candidate queue in the background as the user progresses, incorporating real-time signals (skips, likes) with low latency by updating a lightweight re-ranking layer rather than recomputing the full recommendation from scratch on every signal.

Common mistakes

Calling the recommendation model synchronously right when the current track ends (introduces audible lag/dead air, a critical product failure for a media app); not incorporating real-time signals like skips into the queue at all (feels unresponsive/dumb to the user).

Follow-up question

How would you incorporate a 'user just hit skip' signal into the queue within the next few seconds without recomputing everything?

Evaluation criteria

Explicitly designs for zero-latency playback via pre-fetching/async computation, and addresses incorporating real-time signals cheaply -- doesn't propose synchronous on-demand inference.

You're on-call and get paged: playback is failing for 5% of users in one region, but all service health dashboards show green. Walk me through your triage.

Difficulty: Medium-Hard | Topic: Case Study / Production Incident

Why the interviewer asks it

Spotify's case-study rounds specifically test structured incident triage and communication, not just technical depth.

Ideal thought process

Green dashboards + real user impact usually means the monitoring itself has a blind spot -- narrow by dimension (region, device, app version, network type) rather than re-checking the same aggregate metrics.

Ideal answer

First confirm the signal is real (check raw error logs/traces for that region, not just aggregate dashboards, since a 5% regional failure can be averaged away in a global metric). Segment by likely dimensions: is it one CDN/edge PoP, one app version, one device OS, or one ISP/network type in that region. Check recent deploys/config changes scoped to that region specifically. Communicate status/ETA to stakeholders on a fixed cadence even before root cause is found, rather than going silent while investigating.

Common mistakes

Trusting the green dashboard and assuming user reports are noise (dashboards can easily miss narrow regional/segment-specific failures if they only show global aggregates); not segmenting systematically (just staring at the same dashboards longer); no mention of stakeholder communication during triage.

Follow-up question

The dashboards are green because they only alert on global error rate above 1%. How would you redesign alerting to catch this class of issue earlier?

Evaluation criteria

Immediately questions whether the aggregate metric is hiding a segment-specific problem, and proposes systematic segmentation -- doesn't just restate 'check the logs' vaguely.

Behavioral Questions (STAR)

Tell me about a time your autonomous team (squad) made a decision that conflicted with what another team wanted. How was it resolved?

Situation

The conflicting priorities and the autonomy tension.

Action

How you navigated it -- direct negotiation, escalation, data-driven argument.

Result

The resolution and the relationship afterward.

Evaluation criteria

Spotify's squad model deliberately gives teams autonomy, which creates real cross-team friction -- this question checks whether you can resolve it without needing a manager to intervene every time.

Describe a time you received feedback in a retro or 1:1 that changed how you worked with your team.

Situation

The feedback and the context it came up in.

Action

How you processed and acted on it.

Result

The concrete change and its effect on team dynamics.

Evaluation criteria

Spotify's culture leans heavily on retros and continuous feedback loops -- candidates who describe feedback as threatening rather than useful are a weaker cultural fit signal here.

AI Mock Interview Dataset (RAG-Ready)

Consolidated table across all 11 companies in this volume — columns map directly to company_playbook_chunks metadata fields for retrieval.

Company	Role	Round	Question / Follow-up	Skills Tested / Ideal Answer	Diff.	Tags
Oracle	SDE-2 (Database/Cloud)	Technical Onsite	Composite index / query plan optimization for a slow filtered query	Index column ordering (equality-then-range), EXPLAIN PLAN verification, cardinality reasoning	Medium	sql, indexing, oracle-db, query-optimization
Oracle	SDE-2 (Java Platform)	Technical Onsite	synchronized vs ReentrantLock vs ConcurrentHashMap tradeoffs	JVM intrinsic locks, explicit lock APIs, lock striping in concurrent collections	Medium	java, concurrency, jvm
Oracle	SDE-3 (OCI)	System Design	Design a DB client connection pool for thousands of internal services	Bounded pool, leak detection, health-check/eviction, timeout-based acquisition	Hard	system-design, connection-pooling, oci
Alibaba Cloud	SDE-2	Technical Round 1	Shortest connection path in a large social graph	BFS for unweighted shortest path, bidirectional BFS for scale	Medium	graphs, bfs, algorithms
Alibaba Cloud	SDE-2 (Cloud Infra)	Technical Round 2	Consistent hashing for distributed caches/load balancers	Hash ring, virtual nodes, minimal key movement on scale events	Medium	distributed-systems, consistent-hashing, load-balancing
Alibaba Cloud	SDE-3	System Design	Design a global CDN edge-caching layer	Geo-routing, request coalescing on cache miss, origin shielding, purge/invalidation	Hard	system-design, cdn, caching
Amazon	SDE-2	Onsite Coding	Real-time top-K most purchased products in a sliding hour window	Time-bucketed sliding window, min-heap top-K, count-min sketch at extreme scale	Hard	streaming, top-k, heaps
Amazon	SDE-2	Onsite Coding + LP	Fix double-charging on checkout retries after timeouts	Idempotency key, atomic dedup check, unique DB constraint	Medium	idempotency, distributed-systems, payments
Amazon	SDE-3	System Design	Design Amazon Locker pickup system	Inventory reservation, single-use time-boxed codes, secure server-side validation, reroute fallback	Hard	system-design, iot, security
Amazon	All levels	Bar Raiser (Behavioral)	Ownership LP: fixed a problem outside your responsibility	Closed-loop fix beyond reporting the issue; systemic prevention, not one-off patch	N/A	behavioral, leadership-principles, ownership
Amazon	All levels	Bar Raiser (Behavioral)	Bias for Action LP: decided fast with incomplete data	Reversibility ('two-way door') reasoning before acting under time pressure	N/A	behavioral, leadership-principles, bias-for-action
Amazon	All levels	Bar Raiser (Behavioral)	Disagree and Commit LP: disagreed but executed anyway	Data-backed disagreement pre-decision, genuine commitment post-decision (no relitigating)	N/A	behavioral, leadership-principles, disagree-and-commit
Flipkart	SDE-2	DSA Round	Max profit with at most two stock transactions	State-machine DP (buy1/sell1/buy2/sell2), O(n) single pass, generalizes to K transactions	Medium	dynamic-programming, arrays
Flipkart	SDE-2	Machine Coding	Implement a working per-user rate limiter	Token bucket per user, per-key locking (not global), pluggable algorithm behind interface	Medium-Hard	machine-coding, lld, concurrency

Company	Role	Round	Question / Follow-up	Skills Tested / Ideal Answer	Diff.	Tags
Flipkart	SDE-3	System Design	Cart/inventory system for a flash sale (1M users, 10K units)	Atomic conditional inventory decrement, virtual waiting room/throttling, no read-then-write races	Hard	system-design, e-commerce, concurrency
Microsoft	SDE-2	Onsite Coding	Streaming median via two heaps	Balanced max-heap/min-heap pair, $O(\log n)$ insert, $O(1)$ median lookup	Medium-Hard	heaps, data-structures
Microsoft	SDE-2	LLD Round	Design a parking lot system (levels, vehicle types, dynamic pricing)	Interfaces for allocation strategy and pricing strategy decoupled from core entities	Medium	lld, oop, design-patterns
Microsoft	Senior SDE	System Design	Aggregate health/metrics dashboard for thousands of services	Pull-based polling, bulkheaded failures (stale vs down), serve from store not live calls	Hard	system-design, observability, monitoring
Microsoft	All levels	As Appropriate (Behavioral)	Growth mindset: hard feedback you acted on	Genuine reflection and concrete behavior change, not defensiveness	N/A	behavioral, growth-mindset
Nvidia	SWE (CUDA/Driver)	Technical Onsite	Explain warp divergence and its performance impact	SIMT lockstep execution, path masking/serialization, warp-uniform branching mitigation	Medium	cuda, gpu, parallel-computing
Nvidia	SWE (Perf/HPC)	Technical Onsite	Optimize a memory-bandwidth-bound matrix multiply kernel	Roofline model, shared-memory tiling for data reuse, memory coalescing	Hard	cuda, performance, roofline-model
Nvidia	Senior SWE (ML Infra)	System Design	Schedule ML training jobs across heterogeneous-memory GPU cluster	Memory-aware bin-packing, per-class queues, preemption/checkpointing for priority jobs	Hard	system-design, ml-infra, scheduling
Tesla	SWE (Fleet/Telemetry)	Technical Onsite	Detect unexpected vehicle stop from out-of-order telemetry	Event-time windowing with watermarks, distinguishing legitimate stops from anomalies	Medium-Hard	streaming, real-time-systems
Tesla	Senior SWE (Firmware/OTA)	System Design	Design safe OTA update system for millions of vehicles	A/B partition rollback, health-checked activation, canary/staged rollout	Hard	system-design, ota, safety-critical
OpenAI	SWE (Infra)	Technical Onsite	Resumable checkpoint system for multi-TB dataset processing	Durable granular checkpoints, idempotent shard reprocessing, resume-not-restart	Medium-Hard	data-engineering, checkpointing
OpenAI	SWE (Applied/Infra)	Technical Onsite	Why attention scales quadratically with sequence length	All-pairs $O(n^2)$ compute/memory, sparse/local attention and FlashAttention as distinct mitigations	Medium	transformers, attention, ml-concepts
OpenAI	Senior SWE (Platform)	System Design	Fair-usage rate limiting for an expensive-to-serve model API	Compute-cost-based token budget (not request count), tiered limits, priority queuing under scarcity	Hard	system-design, api-platform, rate-limiting
Broadcom	SWE (Networking)	Technical Phone Screen	TCP vs UDP and when to choose UDP	Handshake/overhead tradeoff, real-time use cases where staleness beats retransmission	Easy-Medium	networking, tcp-udp
Broadcom	SWE (Driver/Kernel)	Technical Onsite	Debug intermittent packet drops under high load	Reliable load reproduction first, race conditions/buffer exhaustion as likely root cause, tracing tools	Hard	debugging, os, drivers
Broadcom	Senior SWE (Networking Software)	System Design	Config push to thousands of switches without partial-failure inconsistency	Two-phase stage/commit, explicit handling of unreachable nodes, automatic rollback	Hard	system-design, networking, distributed-config

Company	Role	Round	Question / Follow-up	Skills Tested / Ideal Answer	Diff.	Tags
Adobe	SDE-2	Technical Onsite	Detect and locate the start of a cycle in a linked list	Floyd's slow/fast pointers, second-phase reset-and-advance math for cycle start	Medium	linked-lists, algorithms
Adobe	Senior SDE (Creative/Document Cloud)	System Design	Real-time collaborative document/design editor backend	CRDT/OT for conflict-free merges, delta-based sync, offline queue and reconciliation	Hard	system-design, crdt, real-time-collaboration
Adobe	Senior SDE (Platform)	System Design	Thumbnail generation/serving pipeline for millions of large media assets	Eager-vs-lazy generation split, request coalescing on cache miss, cost-segregated worker pools	Hard	system-design, media-processing, caching
Spotify	SWE (Backend)	Technical Screen	Implement true shuffle with no repeats until exhaustion, resumable across restarts	Fisher-Yates permutation, persisted order + cursor (not just a boolean flag)	Medium	algorithms, product-engineering
Spotify	Senior SWE (Personalization)	System Design	Real-time next-track recommendation without playback lag	Async pre-fetched candidate queue, background refresh, cheap re-ranking on real-time signals	Hard	system-design, recommendations, latency
Spotify	SWE (SRE/Backend)	Case Study	Triage a regional playback failure invisible to global dashboards	Verify raw signal over aggregate dashboards, systematic segmentation by region/device/version	Medium-Hard	incident-response, observability, case-study